Level 5

Constraints & Keys

SQL Topic Explanations, Examples and Practice Solutions

All SQL is standard and works in MySQL 8 and PostgreSQL. Where they differ, a note is given.

Topics covered

1. Primary Key • 2. Foreign Key • 3. NOT NULL • 4. UNIQUE • 5. DEFAULT • 6. CHECK • 7. Composite Key • 8. Candidate Key • 9. Super Key • 10. Relationships • 11. One-to-One • 12. One-to-Many • 13. Many-to-Many • 14. Referential Integrity • 15. ON DELETE • 16. ON UPDATE

Part 1: Topic Explanations

1. Primary Key (PK)

A column (or set of columns) that **uniquely identifies each row**. It cannot be NULL, cannot repeat, and a table can have **only one**.

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name       VARCHAR(50)
);
```

2. Foreign Key (FK)

A column that **refers to the primary key of another table**. It creates the link between tables and only allows values that exist in the parent table (or NULL).

```
CREATE TABLE departments (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

CREATE TABLE employees (
    emp_id  INT PRIMARY KEY,
    name    VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

departments is the **parent** table and **employees** is the **child** table.

3. NOT NULL

The column **must have a value**. Empty (NULL) is not allowed.

```
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    name    VARCHAR(50) NOT NULL
);
INSERT INTO users VALUES (1, NULL);   -- ERROR
```

4. UNIQUE

All values in the column must be **different**. Unlike a primary key, a table can have many UNIQUE columns, and they usually allow NULL (multiple NULLs are permitted in MySQL and PostgreSQL).

```
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    email   VARCHAR(100) UNIQUE
);
```

5. DEFAULT

Supplies a value automatically when none is given during INSERT.

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    status   VARCHAR(20) DEFAULT 'Pending'
);
INSERT INTO orders (order_id) VALUES (1);   -- status = 'Pending'
```

6. CHECK

Validates data with a condition. The row is rejected if the condition is false. (MySQL enforces CHECK from version 8.0.16; older versions ignore it.)

```
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    price      DECIMAL(10,2) CHECK (price > 0),
    stock      INT CHECK (stock >= 0)
);
INSERT INTO products VALUES (1, -5, 10);   -- ERROR: violates CHECK
```

7. Composite Key

A primary key made of **two or more columns together**. Individual columns may repeat, but the **combination** must be unique.

```
CREATE TABLE order_items (
    order_id   INT,
    product_id INT,
    quantity   INT,
    PRIMARY KEY (order_id, product_id)
);
```

Order 1 can contain products 10 and 11, but (1, 10) cannot appear twice.

8. Candidate Key

Any column or set of columns that could serve as the primary key: unique, non-null, and minimal (no unnecessary columns).

Example: in `students(student_id, email, phone, name)`, both **student_id** and **email** are candidate keys. You pick **one** as the primary key. The others become **alternate keys** (usually defined with UNIQUE).

9. Super Key

Any set of columns that uniquely identifies a row, even if it contains extra columns. Every candidate key is a super key, but not every super key is a candidate key.

Column set	Super key?	Candidate key?
{student_id}	Yes	Yes
{student_id, name}	Yes	No (name is unnecessary)
{email, phone}	Yes	No (extra column)

> **Relationship:** Super keys contain Candidate keys, which contain the Primary key (Primary Key is one of the Candidate Keys, which are a subset of Super Keys).

10. Relationships

Tables connect to each other through keys. There are three types, covered next.

11. One-to-One (1:1)

One row in A matches **at most one** row in B. Implemented with an FK that is also UNIQUE (or the same PK).

```
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    name    VARCHAR(50)
);

CREATE TABLE user_profiles (
    profile_id INT PRIMARY KEY,
    user_id    INT UNIQUE,                -- UNIQUE makes it 1:1
    bio        VARCHAR(200),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

12. One-to-Many (1:N)

One row in A matches **many** rows in B. The FK goes on the **"many"** side. This is the most common relationship.

```
-- One department -> many employees
CREATE TABLE employees (
    emp_id  INT PRIMARY KEY,
    name    VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

13. Many-to-Many (M:N)

Many rows in A match many rows in B. Needs a **junction (bridge) table** holding two FKs, usually forming a composite PK.

```
CREATE TABLE students (student_id INT PRIMARY KEY, name VARCHAR(50));
CREATE TABLE courses  (course_id  INT PRIMARY KEY, title VARCHAR(50));

CREATE TABLE enrollments (
    student_id INT,
    course_id  INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)
);
```

14. Referential Integrity

The rule that **every foreign key value must match an existing primary key** in the parent table (or be NULL). It prevents "orphan" rows, such as an employee in a department that doesn't exist. Foreign keys enforce it automatically.

15. ON DELETE

Defines what happens to child rows when a **parent row is deleted**.

Option	Behavior
CASCADE	Child rows are deleted too
SET NULL	Child FK becomes NULL (column must allow NULL)
SET DEFAULT	Child FK becomes its default value (not supported in MySQL InnoDB)
RESTRICT	Blocks the delete if children exist
NO ACTION	Default; similar to RESTRICT

```
CREATE TABLE employees (
    emp_id  INT PRIMARY KEY,
    name    VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
        ON DELETE CASCADE
);
```

Deleting a department now deletes all its employees.

16. ON UPDATE

Defines what happens to child rows when a **parent's primary key value is changed**. It accepts the same options.

```
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
    ON UPDATE CASCADE
    ON DELETE SET NULL
```

If dept_id 1 changes to 100 in departments, all matching employees.dept_id values update to 100 automatically.

Part 2: Practice Question Solutions

1. Create a table with a primary key

```
CREATE TABLE departments (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(50)
);
```

2. Create a table with a foreign key

```
CREATE TABLE employees (
    emp_id  INT PRIMARY KEY,
    name    VARCHAR(50),
    email   VARCHAR(100),
    salary  INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

3. Add a NOT NULL constraint

```
-- MySQL
ALTER TABLE employees MODIFY name VARCHAR(50) NOT NULL;

-- PostgreSQL
ALTER TABLE employees ALTER COLUMN name SET NOT NULL;
```

4. Add a UNIQUE constraint

```
ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);
```

5. Add a DEFAULT value

```
-- Works in both MySQL and PostgreSQL
ALTER TABLE employees ALTER COLUMN salary SET DEFAULT 30000;
```

6. Add a CHECK constraint

```
ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary >= 0);
```

7. Create a composite primary key

```
CREATE TABLE order_items (
    order_id   INT,
    product_id INT,
    quantity   INT DEFAULT 1,
    PRIMARY KEY (order_id, product_id)
);
```

8. Create a one-to-many relationship

One department has many employees. This is the **departments** and **employees** pair from questions 1 and 2. The FK (dept_id) sits on the "many" side.

9. Create a many-to-many relationship

```
CREATE TABLE students (student_id INT PRIMARY KEY, name VARCHAR(50));
CREATE TABLE courses  (course_id  INT PRIMARY KEY, title VARCHAR(50));

CREATE TABLE enrollments (
    student_id INT,
    course_id  INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)
);
```

10. Insert valid foreign-key records

The parent must exist **before** the child.

```
INSERT INTO departments VALUES (1, 'HR'), (2, 'IT');

INSERT INTO employees (emp_id, name, email, salary, dept_id)
VALUES (101, 'Ravi',  'ravi@mail.com',  40000, 1),
       (102, 'Anita', 'anita@mail.com', 50000, 2);   -- OK
```

11. Try inserting duplicate primary keys

```
INSERT INTO departments VALUES (1, 'Finance');
-- ERROR 1062: Duplicate entry '1' for key 'departments.PRIMARY'
```

12. Try inserting duplicate unique values

```
INSERT INTO employees (emp_id, name, email, salary, dept_id)
VALUES (103, 'Kiran', 'ravi@mail.com', 35000, 1);
-- ERROR 1062: Duplicate entry 'ravi@mail.com' for key 'uq_email'
```

13. Try inserting an invalid foreign key

```
INSERT INTO employees (emp_id, name, email, salary, dept_id)
VALUES (104, 'Sita', 'sita@mail.com', 45000, 99);
-- ERROR 1452: Cannot add or update a child row:
--             a foreign key constraint fails
```

Department 99 does not exist.

14. Test ON DELETE CASCADE

```
-- Recreate the child table with CASCADE
DROP TABLE employees;

CREATE TABLE employees (
    emp_id  INT PRIMARY KEY,
    name    VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
        ON DELETE CASCADE
);

INSERT INTO employees VALUES (101, 'Ravi', 1), (102, 'Anita', 2);

SELECT * FROM employees;           -- 2 rows

DELETE FROM departments WHERE dept_id = 1;

SELECT * FROM employees;           -- only Anita remains; Ravi auto-deleted
```

15. Primary key vs foreign key

Feature	Primary Key	Foreign Key
Purpose	Uniquely identifies a row in its **own** table	Links a row to a row in **another** table
Duplicates	Not allowed	Allowed (many children can share one parent)
NULL	Not allowed	Allowed (unless NOT NULL is set)
Count per table	Only **one** (can be composite)	Can have **many**
Must reference	Nothing	An existing PK/UNIQUE value in the parent table
Example	departments.dept_id	employees.dept_id

In short: the primary key is a row's *identity*, and the foreign key is a *reference* to someone else's identity.

Practice tip: run these in order on a fresh database and deliberately trigger each error. Seeing the real error messages is the fastest way to remember how each constraint behaves.